Part 1 Environment Setup & Imports

```
!pip -q install sentencepiece datasets evaluate openpyxl
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 2.4 MB/s eta 0:00:00
```

```
import os
import re
import random
import warnings
import numpy as np
import pandas as pd

import torch
import torch.nn as nn

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

warnings.filterwarnings("ignore")

print("Libraries Imported Successfully")
```

```
Libraries Imported Successfully
```

```
import transformers
import datasets

print("Transformers :", transformers.__version__)
print("Torch :", torch.__version__)
print("Datasets :", datasets.__version__)
```

```
Transformers : 5.13.1
Torch : 2.11.0+cu128
Datasets : 4.0.0
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device :", device)

if torch.cuda.is_available():
    print("GPU :", torch.cuda.get_device_name(0))
```

```
Device : cuda
GPU : Tesla T4
```

```
SEED = 42

random.seed(SEED)
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

print("Random Seed Set Successfully")
```

```
Random Seed Set Successfully
```

```
CONFIG = {

    "MODEL_NAME": "xlm-roberta-base",

    "MAX_LENGTH": 256,

    "TRAIN_BATCH_SIZE": 32,

    "VALID_BATCH_SIZE": 32,

    "TEST_BATCH_SIZE": 32,

    "EPOCHS": 8,

    "LEARNING_RATE": 1.5e-5,

    "WEIGHT_DECAY": 0.01,

    "TRAIN_SIZE": 0.70,

    "VALID_SIZE": 0.15,

    "TEST_SIZE": 0.15

}

print(CONFIG)
```

```
{'MODEL_NAME': 'xlm-roberta-base', 'MAX_LENGTH': 256, 'TRAIN_BATCH_SIZE': 32,
```

```
os.makedirs("saved_model", exist_ok=True)
os.makedirs("results", exist_ok=True)

print("Project Folders Created")
```

```
Project Folders Created
```

PART 2 Upload Dataset

```

```

```
from google.colab import files

uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
df = pd.read_excel("Multilingual_Airline_Dataset.xlsx")

print("Dataset Loaded Successfully")
```

```
Dataset Loaded Successfully
```

```
print(df.shape)

df.head()
```

```
(6624, 9)
```

	Review_ID	Airline_Name	Rating	Reviews_Text	Recommended	Language	s
0	1	AirAsia India	6.0	✅ Trip Verified | I had booked this fare at a ...	yes	English	
1	2	AirAsia India	6.0	✅ यात्रा सत्यापित | मैंने यह किराया बहत	ves	Hindi	

```
print("="*50)
print("Dataset Shape")
print("="*50)

print(df.shape)
```

```
==================================================
Dataset Shape
==================================================
(6624, 9)
```

```
print(df.columns.tolist())
```

```
['Review_ID', 'Airline_Name', 'Rating', 'Reviews_Text', 'Recommended', 'Langu
```

```
df.head()
```

	Review_ID	Airline_Name	Rating	Reviews_Text	Recommended	Language	S
0	1	AirAsia India	6.0	✅ Trip Verified | I had booked this fare at a ...	yes	English	
1	2	AirAsia India	6.0	✅ यात्रा सत्यापित | मैंने यह किराया बहत	yes	Hindi	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6624 entries, 0 to 6623
Data columns (total 9 columns):
 #   Column                                                          Non-Null Cou
---  ------                                                          ------------
 0   Review_ID                                                       6624 non-nul
 1   Airline_Name                                                    6624 non-nul
 2   Rating                                                          6612 non-nul
 3   Reviews_Text                                                    6624 non-nul
 4   Recommended                                                     6624 non-nul
 5   Language                                                        6624 non-nul
 6   Language, Complaint Category, Sentiment, Severity, Rating,      6624 non-nul
 7   Sentiment                                                       6624 non-nul
 8   Severity                                                        6624 non-nul
dtypes: float64(1), int64(1), object(7)
memory usage: 465.9+ KB
```

Missing Values

```
missing = df.isnull().sum()

missing
```

	0
Review_ID	0
Airline_Name	0
Rating	12
Reviews_Text	0
Recommended	0
Language	0
Language, Complaint Category, Sentiment, Severity, Rating,	0
Sentiment	0
Severity	0

dtype: int64

Duplicate Rows

```
duplicates = df.duplicated().sum()

print("Duplicate Rows :", duplicates)
```

```
Duplicate Rows : 0
```

Remove Duplicates

```
df = df.drop_duplicates().reset_index(drop=True)

print("New Shape :", df.shape)
```

```
New Shape : (6624, 9)
```

Review Length

```
df["Review_Length"] = df["Reviews_Text"].astype(str).apply(len)

df["Review_Length"].describe()
```

	Review_Length
count	6624.000000
mean	613.037742
std	454.591162
min	110.000000
25%	322.000000
50%	479.000000
75%	741.000000
max	3813.000000

dtype: float64

Complaint Category Distribution

```
print("="*60)
print("Complaint Category Distribution")
print("="*60)

print(df["Language, Complaint Category, Sentiment, Severity, Rating,"] .valu
```

```
============================================================
Complaint Category Distribution
============================================================
Language, Complaint Category, Sentiment, Severity, Rating,
Flight Delay           2091
Staff Behavior         1782
Baggage Issue           858
Flight Cancellation     552
Food & Beverages        321
Seat Comfort            306
General Experience      222
Check-in & Boarding     189
Customer Service        114
Refund Issue             99
Booking & Pricing        54
Cleanliness              36
Name: count, dtype: int64
```

Sentiment Distribution

```
print("="*60)
print("Sentiment Distribution")
print("="*60)

print(df["Sentiment"].value_counts())
```

```
============================================================
Sentiment Distribution
============================================================
Sentiment
Negative    4212
Positive    1836
Neutral      576
Name: count, dtype: int64
```

Severity Distribution

```
print("="*60)
print("Severity Distribution")
print("="*60)

print(df["Severity"].value_counts())
```

```
============================================================
Severity Distribution
============================================================
Severity
High      4212
Low       1836
Medium     576
Name: count, dtype: int64
```

Language Distribution

```
print("="*60)
print("Language Distribution")
print("="*60)

print(df["Language"].value_counts())
```

```
============================================================
Language Distribution
============================================================
Language
English    2208
Hindi      2208
Telugu     2208
Name: count, dtype: int64
```

Review Length Statistics

```
print("="*60)
print("Review Length Statistics")
print("="*60)

print("Maximum Length :", df["Review_Length"].max())
print("Minimum Length :", df["Review_Length"].min())
print("Average Length :", round(df["Review_Length"].mean(),2))
```

```
============================================================
Review Length Statistics
============================================================
Maximum Length : 3813
Minimum Length : 110
Average Length : 613.04
```

Class Balance

```
print("="*60)
print("Complaint Category Percentage")
print("="*60)

print(
    round(
        df["Language, Complaint Category, Sentiment, Severity, Rating,"].val
        2
    )
)
```

```
============================================================
Complaint Category Percentage
============================================================
Language, Complaint Category, Sentiment, Severity, Rating,
Flight Delay           31.57
Staff Behavior         26.90
Baggage Issue          12.95
Flight Cancellation     8.33
Food & Beverages        4.85
Seat Comfort            4.62
General Experience      3.35
Check-in & Boarding     2.85
Customer Service        1.72
Refund Issue            1.49
Booking & Pricing       0.82
Cleanliness             0.54
Name: proportion, dtype: float64
```

Dataset Summary

```
print("="*60)

print("DATASET SUMMARY")

print("="*60)

print("Total Samples :", len(df))

print("Total Columns :", len(df.columns))

print("Languages :", df["Language"].nunique())

print("Complaint Categories :", df["Language, Complaint Category, Sentiment,
```

```
print("Sentiment Classes :", df["Sentiment"].nunique())

print("Severity Classes :", df["Severity"].nunique())

print("="*60)
```

```
============================================================
DATASET SUMMARY
============================================================
Total Samples : 6624
Total Columns : 10
Languages : 3
Complaint Categories : 12
Sentiment Classes : 3
Severity Classes : 3
============================================================
```

part 3 Text Preprocessing

```
import re
import string
```

```
def clean_text(text):

    text = str(text)

    # Remove URLs
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)

    # Remove HTML Tags
    text = re.sub(r"<.*?>", "", text)

    # Remove Extra Spaces
    text = re.sub(r"\s+", " ", text)

    # Remove Leading and Trailing Spaces
    text = text.strip()

    return text
```

```
df["Clean_Review"] = df["Reviews_Text"].apply(clean_text)

print("Text Cleaning Completed")
```

```
Text Cleaning Completed
```

```
comparison = pd.DataFrame({

    "Original Review": df["Reviews_Text"].head(5),

    "Clean Review": df["Clean_Review"].head(5)
```

```
})

comparison
```

	Original Review	Clean Review
0	✅ Trip Verified | I had booked this fare at a ...	✅ Trip Verified | I had booked this fare at a ...
1	✅ यात्रा सत्यापित | मैंने यह किराया बहुत रियाय...	✅ यात्रा सत्यापित | मैंने यह किराया बहुत रियाय...
2	✅ ట్రిప్ ధృవీకరించబడింది | నేను BLRకి తిరిగి వ...	✅ ట్రిప్ ధృవీకరించబడింది | నేను BLRకి తిరిగి వ...
	✅ Trip Verified | I travel at least four	✅ Trip Verified | I travel at least four

```
print("Missing Reviews :")

print(df["Clean_Review"].isnull().sum())
```

```
Missing Reviews :
0
```

```
df = df[df["Clean_Review"].str.strip() != ""]

df = df.reset_index(drop=True)

print("Remaining Reviews :", len(df))
```

```
Remaining Reviews : 6624
```

```
df["Token_Length"] = df["Clean_Review"].apply(

    lambda x: len(x.split())

)

df["Token_Length"].describe()
```

	Token_Length
count	6624.000000
mean	104.482337
std	81.761725
min	16.000000
25%	53.000000
50%	80.000000
75%	128.250000
max	757.000000

dtype: float64

```
longest_review = df.loc[
    df["Token_Length"].idxmax(),
    "Clean_Review"
]

print(longest_review)
```

सत्यापित नहीं | इससे पहले कि आप एआई ग्राहक सेवा को कॉल करें - पूरी प्रक्रिया को पूरा करने के

```
shortest_review = df.loc[
    df["Token_Length"].idxmin(),
    "Clean_Review"
]

print(shortest_review)
```

ఢిల్లీ నుంచి చెన్నై వెళ్లాను. మంచి లెగ్‌రూమ్ స్థలం మరియు క్యాబిన్ సిబ్బంది ద్వారా మంచ

```
print("="*60)

print("Average Tokens :", round(df["Token_Length"].mean(),2))

print("Maximum Tokens :", df["Token_Length"].max())

print("Minimum Tokens :", df["Token_Length"].min())

print("="*60)
```

```
============================================================
Average Tokens : 104.48
Maximum Tokens : 757
Minimum Tokens : 16
============================================================
```

```
print(df.head())

print("="*60)

print("Dataset Shape :", df.shape)

print("="*60)
```

```
   Review_ID   Airline_Name  Rating  \
0          1  AirAsia India     6.0
1          2  AirAsia India     6.0
2          3  AirAsia India     6.0
3          4  AirAsia India     1.0
4          5  AirAsia India     1.0

                                        Reviews_Text Recommended Language  \
0  ✅ Trip Verified | I had booked this fare at a ...         yes  English
1  ✅ यात्रा सत्यापित | मैंने यह किराया बहुत रियाय...         yes    Hindi
2  ✅ ట్రిప్ ధృవీకరించబడింది | నేను BLRకి తిరిగి వ...         yes   Telugu
3  ✅ Trip Verified | I travel at least four times...          no  English
4  ✅ यात्रा सत्यापित | मैं महीने में कम से कम चार...          no    Hindi

  Language, Complaint Category, Sentiment, Severity, Rating, Sentiment  \
0                                    Flight Delay           Neutral
1                                    Flight Delay           Neutral
2                                    Flight Delay           Neutral
3                                  Staff Behavior          Negative
4                                  Staff Behavior          Negative

  Severity  Review_Length                                       Clean_Review
0   Medium            976  ✅ Trip Verified | I had booked this fare at a ..
1   Medium            971  ✅ यात्रा सत्यापित | मैंने यह किराया बहुत रियाय...
2   Medium            984  ✅ ట్రిప్ ధృవీకరించబడింది | నేను BLRకి తిరిగి వ...
3     High            543  ✅ Trip Verified | I travel at least four times..
4     High            627  ✅ यात्रा सत्यापित | मैं महीने में कम से कम चार...

   Token_Length
0           188
1           205
2           131
3           103
4           131
============================================================
Dataset Shape : (6624, 12)
============================================================
```

PART 4 Train / Validation / Test Split + Label Encoding

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import joblib
```

keep requied coloumns

```
df = df[
    [
        "Clean_Review",
        "Language, Complaint Category, Sentiment, Severity, Rating,",
        "Sentiment",
        "Severity",
        "Language"
    ]
].copy()

print(df.head())
```

```
                                        Clean_Review  \
0  ✅ Trip Verified | I had booked this fare at a ...
1  ✅ यात्रा सत्यापित | मैंने यह किराया बहुत रियाय...
2  ✅ ట్రిప్ ధృవీకరించబడింది | నేను BLRకి తిరిగి వ...
3  ✅ Trip Verified | I travel at least four times...
4  ✅ यात्रा सत्यापित | मैं महीने में कम से कम चार...

  Language, Complaint Category, Sentiment, Severity, Rating, Sentiment  \
0                                    Flight Delay           Neutral
1                                    Flight Delay           Neutral
2                                    Flight Delay           Neutral
3                                  Staff Behavior          Negative
4                                  Staff Behavior          Negative

  Severity Language
0   Medium  English
1   Medium    Hindi
2   Medium   Telugu
3     High  English
4     High    Hindi
```

Train/Test Split (70% / 30%)

```
train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    random_state=SEED,
    stratify=df["Language, Complaint Category, Sentiment, Severity, Rating,"
)

print("Train Shape :", train_df.shape)
print("Temp Shape :", temp_df.shape)
```

```
Train Shape : (4636, 5)
Temp Shape : (1988, 5)
```

Validation/Test Split (15% / 15%)

```
valid_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=SEED,
```

```
    stratify=temp_df["Language, Complaint Category, Sentiment, Severity, Rat
)

print("Validation Shape :", valid_df.shape)
print("Test Shape :", test_df.shape)
```

```
Validation Shape : (994, 5)
Test Shape : (994, 5)
```

Verify No Data Leakage

```
train_reviews = set(train_df["Clean_Review"])

valid_reviews = set(valid_df["Clean_Review"])

test_reviews = set(test_df["Clean_Review"])

print("Train ∩ Validation :", len(train_reviews & valid_reviews))

print("Train ∩ Test :", len(train_reviews & test_reviews))

print("Validation ∩ Test :", len(valid_reviews & test_reviews))
```

```
Train ∩ Validation : 0
Train ∩ Test : 0
Validation ∩ Test : 0
```

Reset Index

```
train_df = train_df.reset_index(drop=True)
valid_df = valid_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print("Index Reset Completed")
```

```
Index Reset Completed
```

Create Label Encoders (Train Only)

```
category_encoder = LabelEncoder()
sentiment_encoder = LabelEncoder()
severity_encoder = LabelEncoder()
```

Fit Encoders on Training Data

```
category_encoder.fit(train_df["Language, Complaint Category, Sentiment, Seve

sentiment_encoder.fit(train_df["Sentiment"])

severity_encoder.fit(train_df["Severity"])
```

```
print("Encoders Fitted Successfully")
```

```
Encoders Fitted Successfully
```

Transform Labels

```
for dataset in [train_df, valid_df, test_df]:

    dataset["Category_Label"] = category_encoder.transform(
        dataset["Language, Complaint Category, Sentiment, Severity, Rating,"
    )

    dataset["Sentiment_Label"] = sentiment_encoder.transform(
        dataset["Sentiment"]
    )

    dataset["Severity_Label"] = severity_encoder.transform(
        dataset["Severity"]
    )

print("Labels Encoded Successfully")
```

```
Labels Encoded Successfully
```

Number of Classes

```
NUM_CATEGORY = len(category_encoder.classes_)
NUM_SENTIMENT = len(sentiment_encoder.classes_)
NUM_SEVERITY = len(severity_encoder.classes_)

print("Complaint Classes :", NUM_CATEGORY)
print("Sentiment Classes :", NUM_SENTIMENT)
print("Severity Classes :", NUM_SEVERITY)
```

```
Complaint Classes : 12
Sentiment Classes : 3
Severity Classes : 3
```

Save Label Encoders

```
joblib.dump(category_encoder, "saved_model/category_encoder.pkl")
joblib.dump(sentiment_encoder, "saved_model/sentiment_encoder.pkl")
joblib.dump(severity_encoder, "saved_model/severity_encoder.pkl")

print("Label Encoders Saved Successfully")
```

```
Label Encoders Saved Successfully
```

```
train_df.head()
```

	Clean_Review	Language, Complaint Category, Sentiment, Severity, Rating,	Sentiment	Severity	Language	Category_Lab
0	ధృవీకరించబడలేదు | ఉదయం 6.45 గంటలకు మేము ఇంకా 5...	Staff Behavior	Negative	High	Telugu	
1	✅ यात्रा सत्यापित | यह पहली बार है जब मैं	Flight Delay	Negative	High	Hindi	

PART 5 XLM-RoBERTa Tokenizer & DataLoader

```
import torch
from torch.utils.data import Dataset, DataLoader

from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained(
    CONFIG["MODEL_NAME"]
)

print("Tokenizer Loaded Successfully")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
config.json: 100%  615/615 [00:00<00:00, 35.7kB/s]
tokenizer_config.json: 100%  25.0/25.0 [00:00<00:00, 2.00kB/s]
sentencepiece.bpe.model: 100%  5.07M/5.07M [00:00<00:00, 12.4MB/s]
tokenizer.json: 100%  9.10M/9.10M [00:00<00:00, 22.2MB/s]
Tokenizer Loaded Successfully
```

```
sample = train_df["Clean_Review"].iloc[0]

encoding = tokenizer(
    sample,
    truncation=True,
    padding="max_length",
    max_length=CONFIG["MAX_LENGTH"],
    return_tensors="pt"
)

print(sample)

print()
```

```
print("Input IDs Shape :", encoding["input_ids"].shape)

print("Attention Mask Shape :", encoding["attention_mask"].shape)
```

ధృవీకరించబడలేదు | ఉదయం 6.45 గంటలకు మేము ఇంకా 5.10 గంటలకు ఎక్కాల్సిన

```
Input IDs Shape : torch.Size([1, 256])
Attention Mask Shape : torch.Size([1, 256])
```

Custom Dataset

```
class AirlineDataset(Dataset):

    def __init__(self, dataframe, tokenizer, max_length):

        self.data = dataframe.reset_index(drop=True)
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):

        return len(self.data)

    def __getitem__(self, idx):

        review = str(self.data.loc[idx, "Clean_Review"])

        encoding = self.tokenizer(

            review,

            truncation=True,

            padding="max_length",

            max_length=self.max_length,

            return_tensors="pt"

        )

        return {

            "input_ids": encoding["input_ids"].squeeze(0),

            "attention_mask": encoding["attention_mask"].squeeze(0),

            "category_label": torch.tensor(

                self.data.loc[idx, "Category_Label"],

                dtype=torch.long
```

```
            ),
            "sentiment_label": torch.tensor(
                self.data.loc[idx, "Sentiment_Label"],
                dtype=torch.long
            ),
            "severity_label": torch.tensor(
                self.data.loc[idx, "Severity_Label"],
                dtype=torch.long
            )
        }
```

Create Dataset Objects

```
train_dataset = AirlineDataset(
    train_df,
    tokenizer,
    CONFIG["MAX_LENGTH"]
)
valid_dataset = AirlineDataset(
    valid_df,
    tokenizer,
    CONFIG["MAX_LENGTH"]
)
test_dataset = AirlineDataset(
    test_df,
    tokenizer,
    CONFIG["MAX_LENGTH"]
)
print("Datasets Created Successfully")
```

```
Datasets Created Successfully
```

Create DataLoaders

```
train_loader = DataLoader(
    train_dataset,
    batch_size=CONFIG["TRAIN_BATCH_SIZE"],
    shuffle=True,
    num_workers=2,
    pin_memory=True
)
valid_loader = DataLoader(
    valid_dataset,
    batch_size=CONFIG["VALID_BATCH_SIZE"],
    shuffle=False,
    num_workers=2,
    pin_memory=True
)
test_loader = DataLoader(
    test_dataset,
    batch_size=CONFIG["TEST_BATCH_SIZE"],
    shuffle=False,
    num_workers=2,
    pin_memory=True
)
print("DataLoaders Created Successfully")
```

```
DataLoaders Created Successfully
```

Verify Batch

```
batch = next(iter(train_loader))

print(batch.keys())
```

```
dict_keys(['input_ids', 'attention_mask', 'category_label', 'sentiment_label'
```

Verify Tensor Shapes

```
print("Input IDs :", batch["input_ids"].shape)

print("Attention Mask :", batch["attention_mask"].shape)

print("Category :", batch["category_label"].shape)

print("Sentiment :", batch["sentiment_label"].shape)

print("Severity :", batch["severity_label"].shape)
```

```
Input IDs : torch.Size([32, 256])
Attention Mask : torch.Size([32, 256])
Category : torch.Size([32])
Sentiment : torch.Size([32])
Severity : torch.Size([32])
```

PART 6 Multi-Task XLM-RoBERTa Model

```
import torch
import torch.nn as nn

from transformers import AutoModel

print("Libraries Imported Successfully")
```

```
Libraries Imported Successfully
```

Build Multi-Task Model

```
import torch
import torch.nn as nn
from transformers import AutoModel


class MultiTaskXLMRoberta(nn.Module):

    def __init__(
        self,
        model_name,
        num_category,
        num_sentiment,
        num_severity,
        dropout=0.3
```

```
        ):
            super().__init__()

            self.encoder = AutoModel.from_pretrained(model_name)

            hidden_size = self.encoder.config.hidden_size

            self.dropout = nn.Dropout(dropout)

            self.category_head = nn.Linear(hidden_size, num_category)
            self.sentiment_head = nn.Linear(hidden_size, num_sentiment)
            self.severity_head = nn.Linear(hidden_size, num_severity)

        def mean_pooling(self, model_output, attention_mask):

            token_embeddings = model_output.last_hidden_state

            input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embe

            return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.
                input_mask_expanded.sum(1),
                min=1e-9
            )

        def forward(self, input_ids, attention_mask):

            outputs = self.encoder(
                input_ids=input_ids,
                attention_mask=attention_mask
            )

            pooled_output = self.mean_pooling(outputs, attention_mask)

            pooled_output = self.dropout(pooled_output)

            category_logits = self.category_head(pooled_output)
            sentiment_logits = self.sentiment_head(pooled_output)
            severity_logits = self.severity_head(pooled_output)

            return category_logits, sentiment_logits, severity_logits
```

Create Model

```
    model = MultiTaskXLMRoberta(

        model_name=CONFIG["MODEL_NAME"],

        num_category=NUM_CATEGORY,

        num_sentiment=NUM_SENTIMENT,

        num_severity=NUM_SEVERITY,
```

```
    dropout=0.3

)

print("Model Created Successfully")
```

```
model.safetensors: reconstructing file: 100%                    1.12GB / 1.12GB, 86.5MB/s

model.safetensors: downloading bytes:                                   745MB, 62.7MB/s

Loading weights: 100%                                  199/199 [00:00<00:00, 3932.69it/s]
[transformers] XLMRobertaModel LOAD REPORT from: xlm-roberta-base
Key                         | Status     |  |
----------------------------+------------+--+-
lm_head.layer_norm.bias     | UNEXPECTED |  |
lm_head.dense.weight        | UNEXPECTED |  |
lm_head.bias                | UNEXPECTED |  |
lm_head.dense.bias          | UNEXPECTED |  |
lm_head.layer_norm.weight | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
Model Created Successfully
```

Loss Functions

```
category_loss = nn.CrossEntropyLoss()

sentiment_loss = nn.CrossEntropyLoss()

severity_loss = nn.CrossEntropyLoss()

print("Loss Functions Ready")
```

```
Loss Functions Ready
```

Optimizer

```
optimizer = torch.optim.AdamW(

    model.parameters(),

    lr=CONFIG["LEARNING_RATE"],

    weight_decay=CONFIG["WEIGHT_DECAY"]

)

print("Optimizer Ready")
```

```
Optimizer Ready
```

Scheduler

```
from transformers import get_linear_schedule_with_warmup

total_steps = len(train_loader) * CONFIG["EPOCHS"]

scheduler = get_linear_schedule_with_warmup(

    optimizer,

    num_warmup_steps=int(0.1 * total_steps),

    num_training_steps=total_steps

)

print("Scheduler Ready")
```

```
Scheduler Ready
```

Count Parameters

```
total_params = sum(p.numel() for p in model.parameters())

trainable_params = sum(

    p.numel()

    for p in model.parameters()

    if p.requires_grad

)

print("Total Parameters :", total_params)

print("Trainable Parameters :", trainable_params)
```

```
Total Parameters : 278057490
Trainable Parameters : 278057490
```

Test Forward Pass

```
batch = next(iter(train_loader))

input_ids = batch["input_ids"].to(device)
attention_mask = batch["attention_mask"].to(device)

# Make sure the model is on the same device
model = model.to(device)
model.eval()
```

```
with torch.no_grad():

    category_logits, sentiment_logits, severity_logits = model(
        input_ids=input_ids,
        attention_mask=attention_mask
    )

print("Category Output :", category_logits.shape)
print("Sentiment Output :", sentiment_logits.shape)
print("Severity Output :", severity_logits.shape)
```

```
Category Output : torch.Size([32, 12])
Sentiment Output : torch.Size([32, 3])
Severity Output : torch.Size([32, 3])
```

```
print("Device:", device)

print("Model Device:", next(model.parameters()).device)

print("Input Device:", input_ids.device)

print("Attention Mask Device:", attention_mask.device)
```

```
Device: cuda
Model Device: cuda:0
Input Device: cuda:0
Attention Mask Device: cuda:0
```

model summary

```
print("=" * 60)
print("MODEL SUMMARY")
print("=" * 60)

print("Encoder :", CONFIG["MODEL_NAME"])
print("Complaint Classes :", NUM_CATEGORY)
print("Sentiment Classes :", NUM_SENTIMENT)
print("Severity Classes :", NUM_SEVERITY)
print("Learning Rate :", CONFIG["LEARNING_RATE"])
print("Epochs :", CONFIG["EPOCHS"])

print("=" * 60)
```

```
============================================================
MODEL SUMMARY
============================================================
Encoder : xlm-roberta-base
Complaint Classes : 12
Sentiment Classes : 3
Severity Classes : 3
Learning Rate : 1.5e-05
Epochs : 8
============================================================
```

Mixed Precision (GPU)

```
from torch.amp import GradScaler, autocast

scaler = GradScaler("cuda")

print("GradScaler Initialized")
```

```
GradScaler Initialized
```

Training Function

```
def train_one_epoch(model, loader, optimizer, scheduler):

    model.train()

    total_loss = 0

    cat_correct = 0
    sen_correct = 0
    sev_correct = 0

    total = 0

    for batch in loader:

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)

        category = batch["category_label"].to(device)
        sentiment = batch["sentiment_label"].to(device)
        severity = batch["severity_label"].to(device)

        optimizer.zero_grad()

        with autocast(device_type="cuda"):

            cat_logits, sen_logits, sev_logits = model(
                input_ids=input_ids,
                attention_mask=attention_mask
            )

            loss1 = category_loss(cat_logits, category)
            loss2 = sentiment_loss(sen_logits, sentiment)
            loss3 = severity_loss(sev_logits, severity)

            loss = loss1 + loss2 + loss3

        scaler.scale(loss).backward()

        scaler.step(optimizer)
```

```
            scaler.update()

            scheduler.step()

            total_loss += loss.item()

            cat_correct += (cat_logits.argmax(1) == category).sum().item()
            sen_correct += (sen_logits.argmax(1) == sentiment).sum().item()
            sev_correct += (sev_logits.argmax(1) == severity).sum().item()

            total += category.size(0)

        return (
            total_loss / len(loader),
            cat_correct / total,
            sen_correct / total,
            sev_correct / total
        )
```

Validation Function

```
    def validate(model, loader):

        model.eval()

        total_loss = 0

        cat_correct = 0
        sen_correct = 0
        sev_correct = 0

        total = 0

        with torch.no_grad():

            for batch in loader:

                input_ids = batch["input_ids"].to(device)
                attention_mask = batch["attention_mask"].to(device)

                category = batch["category_label"].to(device)
                sentiment = batch["sentiment_label"].to(device)
                severity = batch["severity_label"].to(device)

                cat_logits, sen_logits, sev_logits = model(
                    input_ids=input_ids,
                    attention_mask=attention_mask
                )

                loss = (
                    category_loss(cat_logits, category)
                    + sentiment_loss(sen_logits, sentiment)
                    + severity_loss(sev_logits, severity)
```

```
            )

            total_loss += loss.item()

            cat_correct += (cat_logits.argmax(1) == category).sum().item()
            sen_correct += (sen_logits.argmax(1) == sentiment).sum().item()
            sev_correct += (sev_logits.argmax(1) == severity).sum().item()

            total += category.size(0)

    return (
        total_loss / len(loader),
        cat_correct / total,
        sen_correct / total,
        sev_correct / total
    )
```

Initialize Training

```
best_loss = float("inf")

history = {
    "train_loss": [],
    "valid_loss": [],
    "category_acc": [],
    "sentiment_acc": [],
    "severity_acc": []
}

print("Training Initialized")
```

```
Training Initialized
```

Train the Model

```
best_loss = float("inf")

for epoch in range(CONFIG["EPOCHS"]):

    print("="*70)
    print(f"Epoch {epoch+1}/{CONFIG['EPOCHS']}")
    print("="*70)

    train_loss, train_cat_acc, train_sent_acc, train_sev_acc = train_one_epo
        model,
        train_loader,
        optimizer,
        scheduler
    )

    valid_loss, valid_cat_acc, valid_sent_acc, valid_sev_acc = validate(
        model,
```

```
            valid_loader
        )

        history["train_loss"].append(train_loss)
        history["valid_loss"].append(valid_loss)

        history["category_acc"].append(valid_cat_acc)
        history["sentiment_acc"].append(valid_sent_acc)
        history["severity_acc"].append(valid_sev_acc)

        print(f"Train Loss      : {train_loss:.4f}")
        print(f"Validation Loss : {valid_loss:.4f}")

        print(f"Category Accuracy  : {valid_cat_acc*100:.2f}%")
        print(f"Sentiment Accuracy : {valid_sent_acc*100:.2f}%")
        print(f"Severity Accuracy  : {valid_sev_acc*100:.2f}%")

        if valid_loss < best_loss:

            best_loss = valid_loss

            torch.save(
                model.state_dict(),
                "saved_model/best_multitask_xlm_roberta.pt"
            )

            print(" Best Model Saved")

    print("\n Training Completed Successfully!")
```

```
Sentiment Accuracy : 90.24%
Severity Accuracy  : 90.44%
 Best Model Saved
================================================================================
Epoch 3/8
================================================================================
Train Loss      : 1.3807
Validation Loss : 1.2207
Category Accuracy  : 80.08%
Sentiment Accuracy : 90.24%
Severity Accuracy  : 90.34%
 Best Model Saved
```

```
Severity Accuracy  : 91.35%
 Best Model Saved
======================================================================
Epoch 6/8
======================================================================
Train Loss      : 0.6818
Validation Loss : 0.9771
Category Accuracy  : 85.11%
Sentiment Accuracy : 91.25%
Severity Accuracy  : 91.25%
 Best Model Saved
======================================================================
Epoch 7/8
======================================================================
Train Loss      : 0.5867
Validation Loss : 0.9718
Category Accuracy  : 85.81%
Sentiment Accuracy : 91.65%
Severity Accuracy  : 91.55%
 Best Model Saved
======================================================================
Epoch 8/8
======================================================================
Train Loss      : 0.5341
Validation Loss : 0.9570
Category Accuracy  : 85.71%
Sentiment Accuracy : 91.65%
Severity Accuracy  : 92.05%
 Best Model Saved

 Training Completed Successfully!
```

Training Summary

```
print("="*70)
print("           TRAINING SUMMARY")
print("="*70)

print(f"Best Validation Loss        : {best_loss:.4f}")

print(f"Best Category Accuracy      : {max(history['category_acc'])*100:.2f}
print(f"Best Sentiment Accuracy     : {max(history['sentiment_acc'])*100:.2f
print(f"Best Severity Accuracy      : {max(history['severity_acc'])*100:.2f}

overall_best = (
    max(history["category_acc"]) +
    max(history["sentiment_acc"]) +
    max(history["severity_acc"])
) / 3

print(f"\nOverall Multi-Task Accuracy : {overall_best*100:.2f}%")

print("="*70)

print("Training Completed Successfully ✅")
```

```
======================================================================
           TRAINING SUMMARY
======================================================================
Best Validation Loss        : 0.9570
Best Category Accuracy      : 85.81%
Best Sentiment Accuracy     : 91.65%
Best Severity Accuracy      : 92.05%

Overall Multi-Task Accuracy : 89.84%
======================================================================
Training Completed Successfully ✅
```

Import Evaluation Libraries

```
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix
)

import numpy as np

print("Evaluation Libraries Imported Successfully")
```

```
Evaluation Libraries Imported Successfully
```

Generate Predictions on the Test Set

```
model.eval()

cat_true = []
cat_pred = []

sen_true = []
sen_pred = []

sev_true = []
sev_pred = []

with torch.no_grad():

    for batch in test_loader:

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)

        category = batch["category_label"].to(device)
        sentiment = batch["sentiment_label"].to(device)
        severity = batch["severity_label"].to(device)

        cat_logits, sen_logits, sev_logits = model(
            input_ids=input_ids,
```

```
                attention_mask=attention_mask
            )

            cat_prediction = torch.argmax(cat_logits, dim=1)
            sen_prediction = torch.argmax(sen_logits, dim=1)
            sev_prediction = torch.argmax(sev_logits, dim=1)

            cat_true.extend(category.cpu().numpy())
            cat_pred.extend(cat_prediction.cpu().numpy())

            sen_true.extend(sentiment.cpu().numpy())
            sen_pred.extend(sen_prediction.cpu().numpy())

            sev_true.extend(severity.cpu().numpy())
            sev_pred.extend(sev_prediction.cpu().numpy())

    print("Prediction Completed Successfully")
```

```
Prediction Completed Successfully
```

Calculate Accuracy

```
cat_acc = accuracy_score(cat_true, cat_pred)
sen_acc = accuracy_score(sen_true, sen_pred)
sev_acc = accuracy_score(sev_true, sev_pred)

overall_acc = (cat_acc + sen_acc + sev_acc) / 3

print("="*70)
print("TEST ACCURACY")
print("="*70)

print(f"Complaint Category : {cat_acc*100:.2f}%")
print(f"Sentiment          : {sen_acc*100:.2f}%")
print(f"Severity           : {sev_acc*100:.2f}%")
print(f"\nOverall Accuracy   : {overall_acc*100:.2f}%")
```

```
======================================================================
TEST ACCURACY
======================================================================
Complaint Category : 84.41%
Sentiment          : 91.45%
Severity           : 91.35%

Overall Accuracy   : 89.07%
```

Precision, Recall & F1 Score

```
for name, y_true, y_pred in [

    ("Complaint Category", cat_true, cat_pred),

    ("Sentiment", sen_true, sen_pred),
```

```
    ("Severity", sev_true, sev_pred)

]:

    precision, recall, f1, _ = precision_recall_fscore_support(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    print("="*70)
    print(name)
    print("="*70)

    print(f"Precision : {precision:.4f}")
    print(f"Recall    : {recall:.4f}")
    print(f"F1-Score  : {f1:.4f}")
```

```
======================================================================
Complaint Category
======================================================================
Precision : 0.8312
Recall    : 0.8441
F1-Score  : 0.8323
======================================================================
Sentiment
======================================================================
Precision : 0.9069
Recall    : 0.9145
F1-Score  : 0.9090
======================================================================
Severity
======================================================================
Precision : 0.9063
Recall    : 0.9135
F1-Score  : 0.9083
```

Classification Reports

```
print("="*70)
print("Complaint Category Report")
print("="*70)

print(classification_report(
    cat_true,
    cat_pred,
    target_names=category_encoder.classes_
))

print("="*70)
print("Sentiment Report")
print("="*70)
```

```
print(classification_report(
    sen_true,
    sen_pred,
    target_names=sentiment_encoder.classes_
))

print("="*70)
print("Severity Report")
print("="*70)

print(classification_report(
    sev_true,
    sev_pred,
    target_names=severity_encoder.classes_
))
```

```
======================================================================
Complaint Category Report
======================================================================
                     precision    recall  f1-score   support

      Baggage Issue       0.84      0.95      0.89       128
  Booking & Pricing       0.00      0.00      0.00         8
Check-in & Boarding       0.67      0.43      0.52        28
        Cleanliness       0.00      0.00      0.00         6
   Customer Service       0.40      0.12      0.18        17
Flight Cancellation       0.84      0.89      0.87        83
       Flight Delay       0.94      0.85      0.89       314
   Food & Beverages       0.79      0.96      0.87        48
 General Experience       0.47      0.65      0.54        34
       Refund Issue       0.50      0.27      0.35        15
       Seat Comfort       0.76      0.91      0.83        46
     Staff Behavior       0.87      0.93      0.90       267

           accuracy                           0.84       994
          macro avg       0.59      0.58      0.57       994
       weighted avg       0.83      0.84      0.83       994

======================================================================
Sentiment Report
======================================================================
              precision    recall  f1-score   support

    Negative       0.95      0.98      0.97       627
     Neutral       0.66      0.47      0.55        94
    Positive       0.89      0.92      0.90       273

    accuracy                           0.91       994
   macro avg       0.83      0.79      0.81       994
weighted avg       0.91      0.91      0.91       994

======================================================================
Severity Report
======================================================================
              precision    recall  f1-score   support

        High       0.95      0.98      0.97       627
         Low       0.89      0.91      0.90       273
```

```
      Medium       0.66      0.48      0.56        94

    accuracy                           0.91       994
   macro avg       0.83      0.79      0.81       994
weighted avg       0.91      0.91      0.91       994
```

Save Final Model

```
torch.save(model.state_dict(), "saved_model/final_multitask_model.pt")

print("="*70)
print("FINAL MODEL SAVED SUCCESSFULLY")
print("="*70)

print("Model Path : saved_model/final_multitask_model.pt")
```

```
======================================================================
FINAL MODEL SAVED SUCCESSFULLY
======================================================================
Model Path : saved_model/final_multitask_model.pt
```

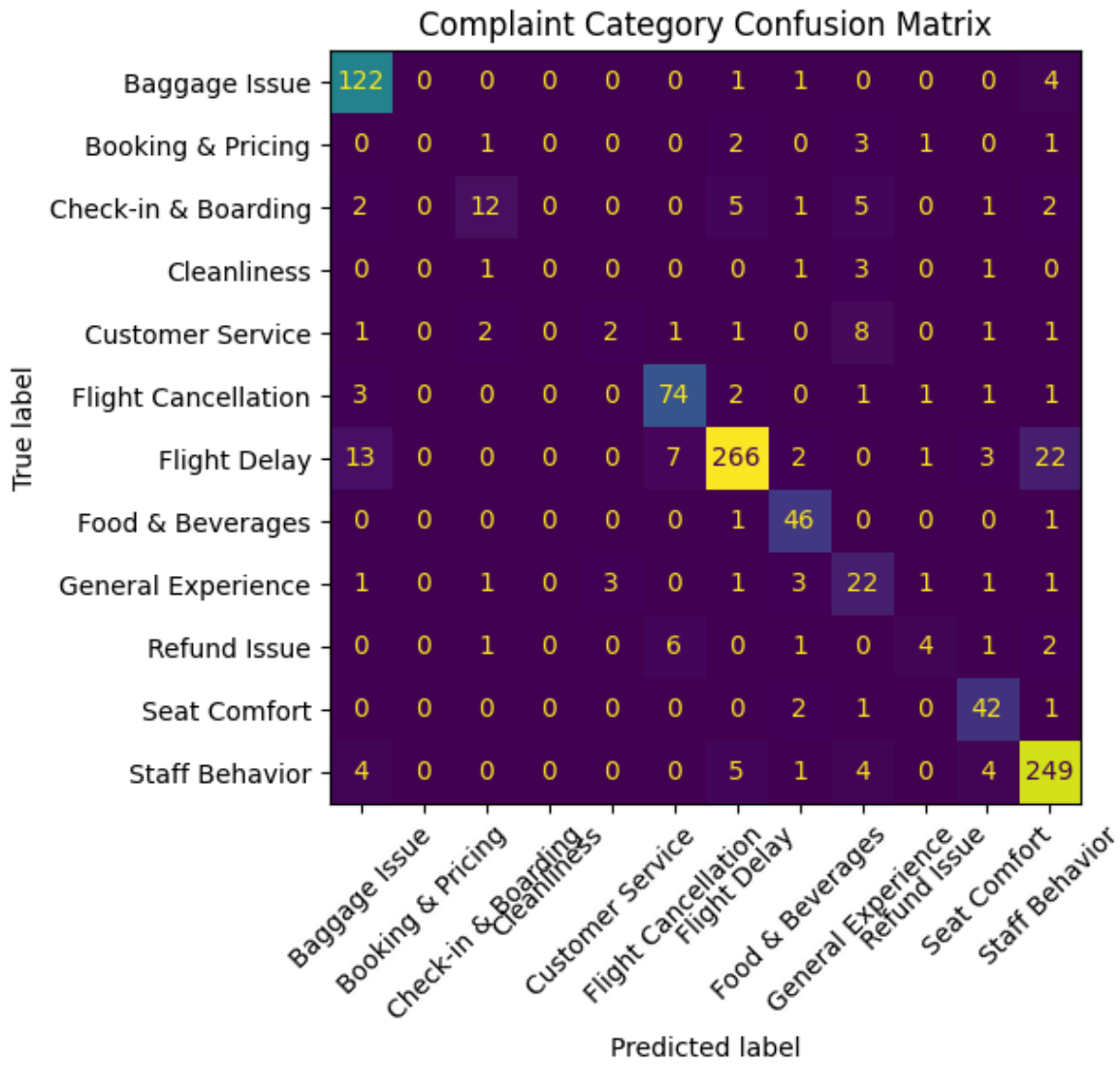
Import Visualization Libraries

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

print("Visualization Libraries Imported Successfully")
```

```
Visualization Libraries Imported Successfully
```

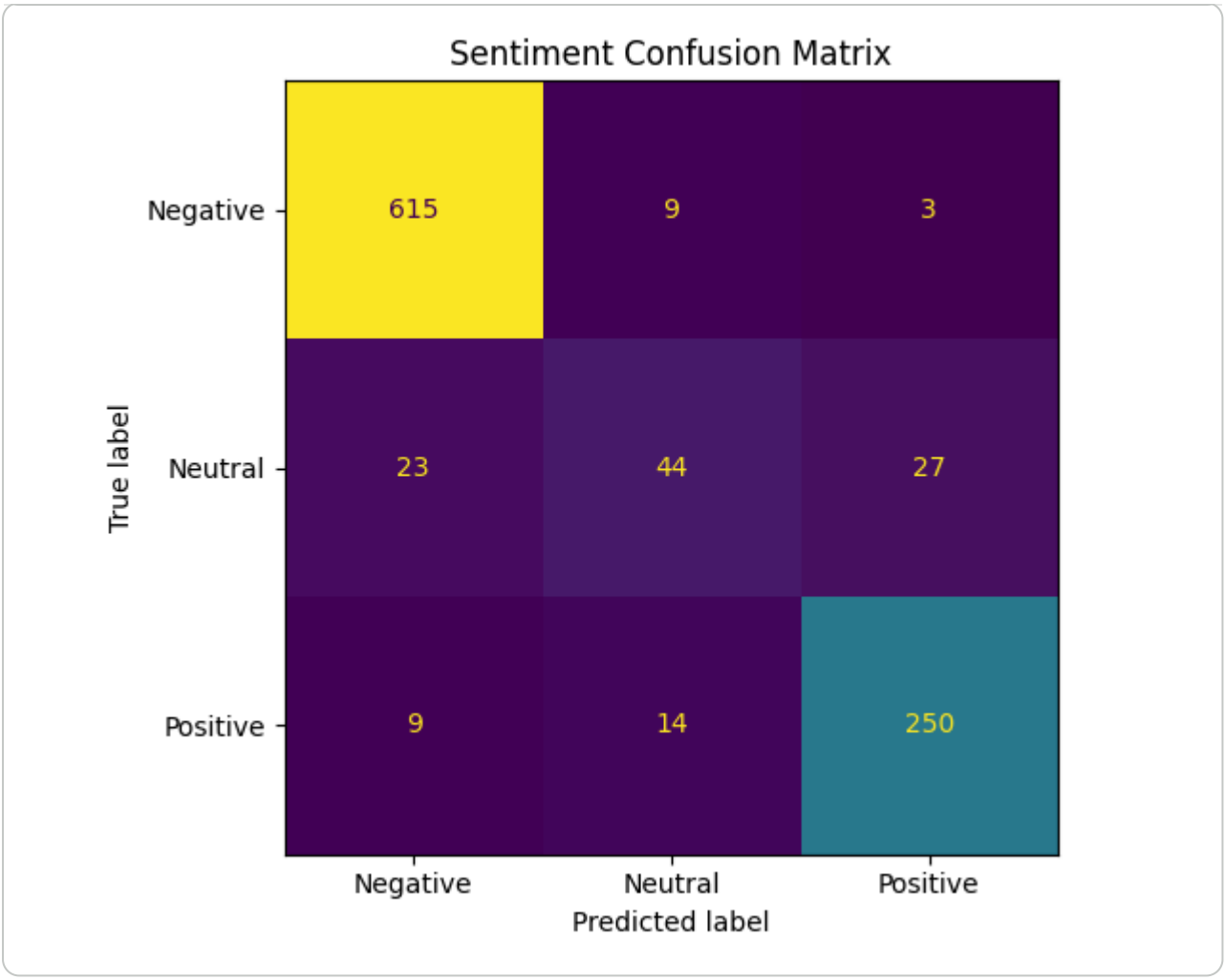
Complaint Category Confusion Matrix

```
cm = confusion_matrix(cat_true, cat_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=category_encoder.classes_
)

fig, ax = plt.subplots(figsize=(8,6))
disp.plot(ax=ax, xticks_rotation=45, colorbar=False)

plt.title("Complaint Category Confusion Matrix")
plt.tight_layout()
plt.show()
```



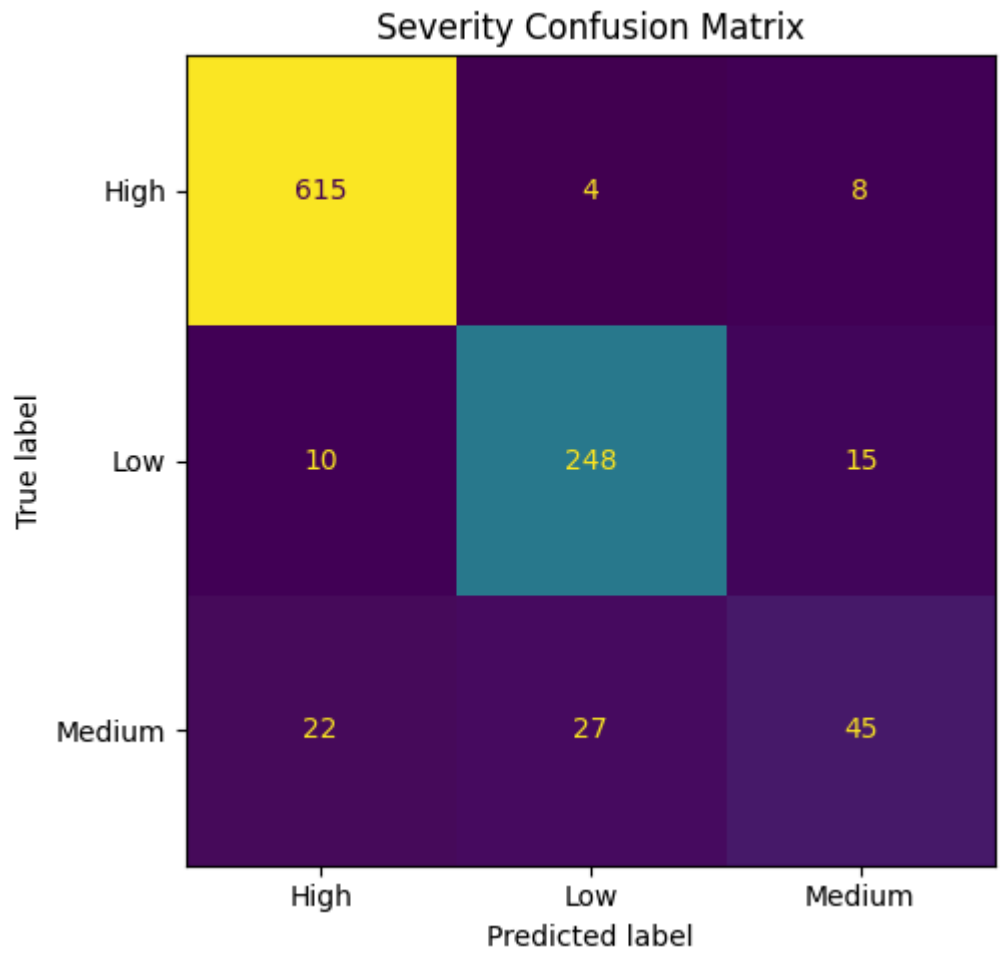
Sentiment Confusion Matrix

```
cm = confusion_matrix(sen_true, sen_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=sentiment_encoder.classes_
)

fig, ax = plt.subplots(figsize=(6,5))
disp.plot(ax=ax, colorbar=False)

plt.title("Sentiment Confusion Matrix")
plt.tight_layout()
plt.show()
```



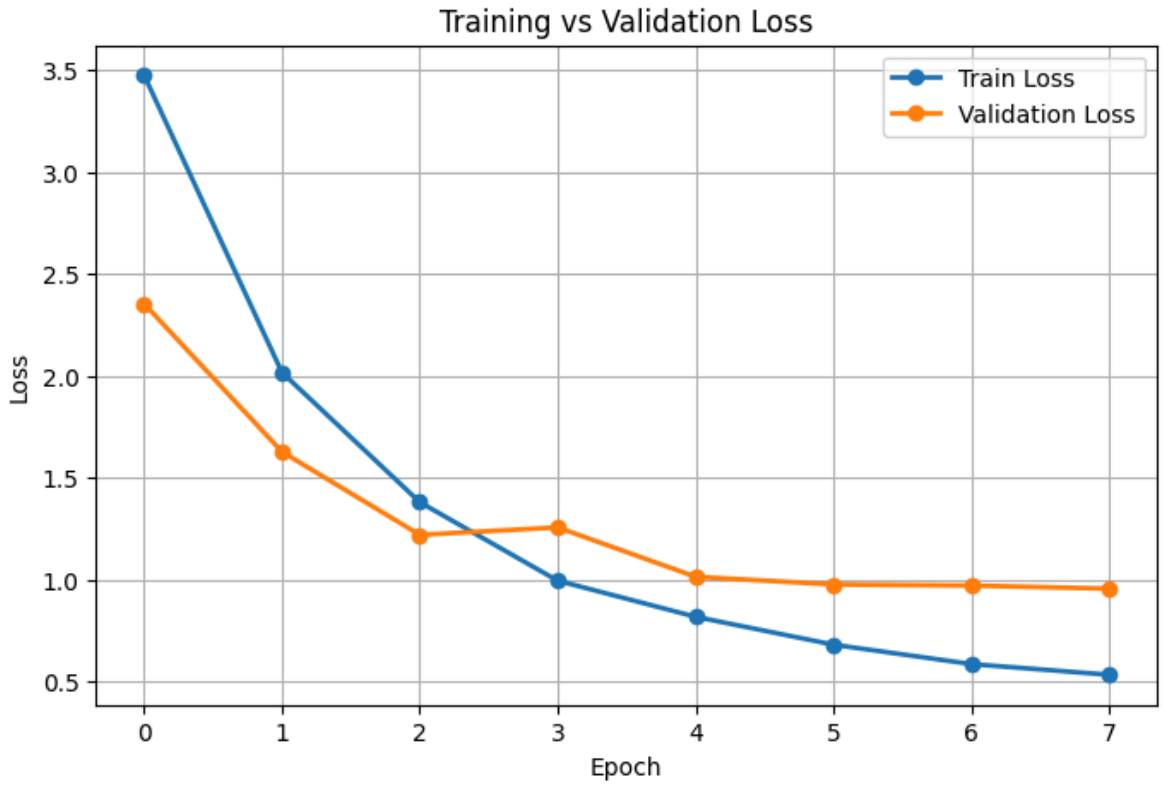
Severity Confusion Matrix

```
Scm = confusion_matrix(sev_true, sev_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=severity_encoder.classes_
)

fig, ax = plt.subplots(figsize=(6,5))
disp.plot(ax=ax, colorbar=False)

plt.title("Severity Confusion Matrix")
plt.tight_layout()
plt.show()
```



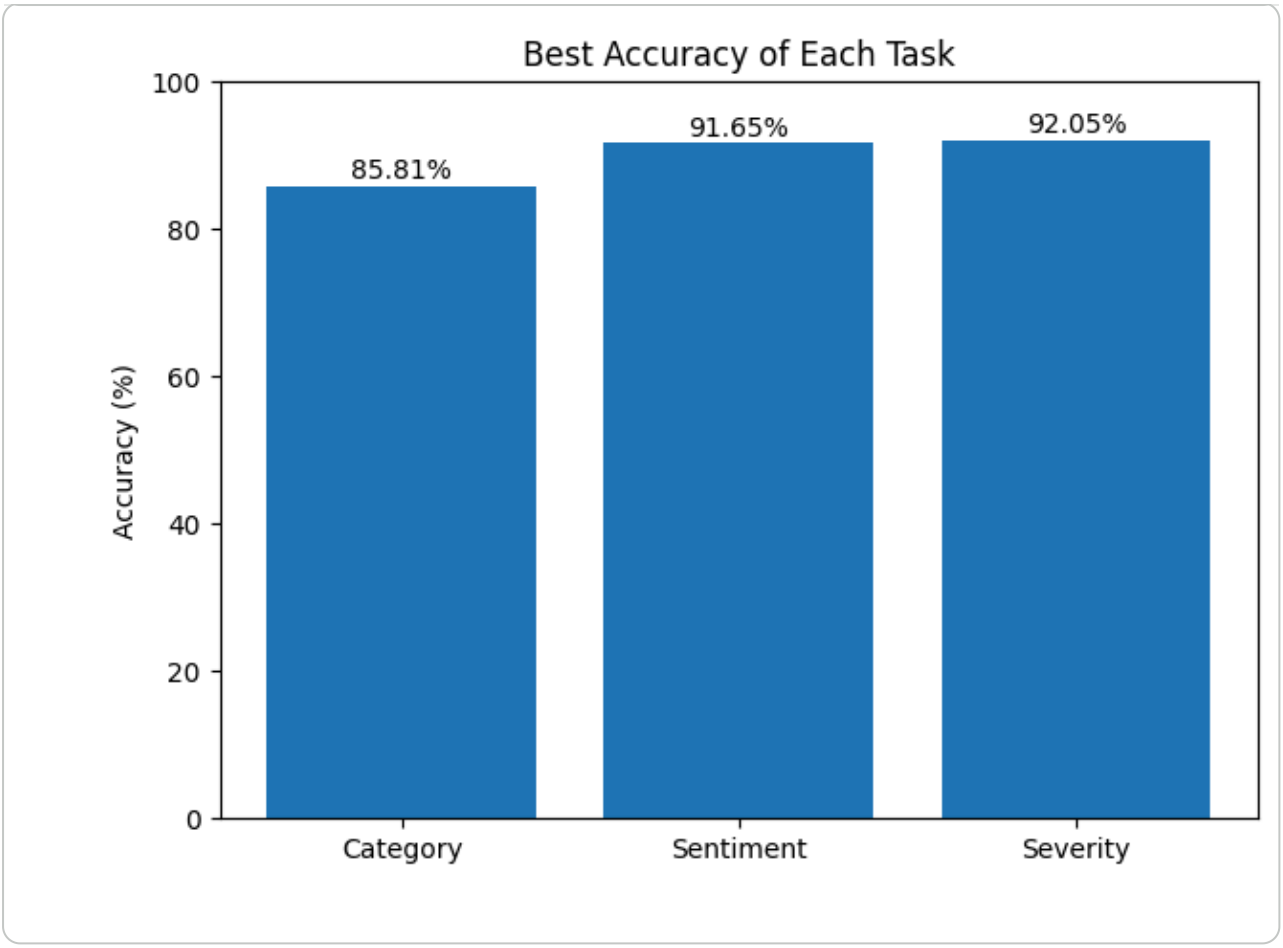
Training vs Validation Loss

```
plt.figure(figsize=(8,5))

plt.plot(history["train_loss"], marker="o", linewidth=2, label="Train Loss")
plt.plot(history["valid_loss"], marker="o", linewidth=2, label="Validation L

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Accuracy Comparison

```
labels = ["Category", "Sentiment", "Severity"]

accuracy = [
    max(history["category_acc"])*100,
    max(history["sentiment_acc"])*100,
    max(history["severity_acc"])*100
]

plt.figure(figsize=(7,5))

plt.bar(labels, accuracy)

plt.ylabel("Accuracy (%)")
plt.ylim(0,100)

plt.title("Best Accuracy of Each Task")

for i, value in enumerate(accuracy):
    plt.text(i, value+1, f"{value:.2f}%", ha="center")

plt.show()
```



Overall Accuracy Summary

```
overall = np.mean(accuracy)

print("="*60)
print("MODEL PERFORMANCE SUMMARY")
print("="*60)

print(f"Complaint Accuracy : {accuracy[0]:.2f}%")
print(f"Sentiment Accuracy : {accuracy[1]:.2f}%")
print(f"Severity Accuracy  : {accuracy[2]:.2f}%")

print("-"*60)

print(f"Overall Accuracy : {overall:.2f}%")

print("="*60)
```

```
============================================================
MODEL PERFORMANCE SUMMARY
============================================================
Complaint Accuracy : 85.81%
Sentiment Accuracy : 91.65%
Severity Accuracy  : 92.05%
------------------------------------------------------------
Overall Accuracy : 89.84%
============================================================
```

Output()

```
def predict_review(review):

    model.eval()

    review = clean_text(review)

    encoding = tokenizer(

        review,

        truncation=True,

        padding="max_length",

        max_length=CONFIG["MAX_LENGTH"],

        return_tensors="pt"

    )

    input_ids = encoding["input_ids"].to(device)
    attention_mask = encoding["attention_mask"].to(device)

    with torch.no_grad():

        cat_logits, sen_logits, sev_logits = model(

            input_ids=input_ids,

            attention_mask=attention_mask

        )

    cat_pred = torch.argmax(cat_logits, dim=1).cpu().item()
    sen_pred = torch.argmax(sen_logits, dim=1).cpu().item()
    sev_pred = torch.argmax(sev_logits, dim=1).cpu().item()

    category = category_encoder.inverse_transform([cat_pred])[0]
    sentiment = sentiment_encoder.inverse_transform([sen_pred])[0]
    severity = severity_encoder.inverse_transform([sev_pred])[0]

    return category, sentiment, severity
```

```
review = input("Enter Airline Review:\n\n")
```

```
Enter Airline Review:

     "Flight was delayed for 6 hours and nobody helped us.",
```

```
category, sentiment, severity = predict_review(review)

print("Prediction Completed Successfully")
```

```
Prediction Completed Successfully
```

RECOMMENDATION FUNCTION

```
def recommendation(severity):

    severity = severity.lower()

    if severity == "high":
        return "Immediate attention required. Escalate to airline management

    elif severity == "medium":
        return "Customer support should contact the passenger."

    else:
        return "Record the feedback for future service improvement."
```

Final Airline Passenger Feedback Report

```
print("="*75)
print("        AIRLINE PASSENGER FEEDBACK ANALYSIS REPORT")
print("="*75)

print("\nPassenger Review\n")
print(review)

print("\nPredicted Complaint Category")
print(category)

print("\nPredicted Sentiment")
print(sentiment)

print("\nPredicted Severity")
print(severity)

print("\nRecommended Action")
print(recommendation(severity))

print("="*75)
```

```
===========================================================================
        AIRLINE PASSENGER FEEDBACK ANALYSIS REPORT
===========================================================================

Passenger Review

     "Flight was delayed for 6 hours and nobody helped us.",

Predicted Complaint Category
```

```
Flight Delay

Predicted Sentiment
Negative

Predicted Severity
High

Recommended Action
Immediate attention required. Escalate to airline management.
===========================================================================
```

```
print("="*75)
print("        AIRLINE PASSENGER FEEDBACK ANALYSIS REPORT")
print("="*75)

print("\nPassenger Review\n")
print(review)

print("\nPredicted Complaint Category")
print(category)

print("\nPredicted Sentiment")
print(sentiment)

print("\nPredicted Severity")
print(severity)

print("\nRecommended Action")
print(recommendation(severity))

print("="*75)
```

```
===========================================================================
        AIRLINE PASSENGER FEEDBACK ANALYSIS REPORT
===========================================================================
```